Optimal Classification Trees: Main Results and Additional Studies

Charlène Krick and Agathe Pascal

March 2026

1 Introduction

This report compares five datasets. Three datasets are given in the project statement: `iris`, `seeds`, and `wine`. We add two datasets: `banknote` and `wdbc`. The goal is to compare the exact formulation F with two lighter approaches based on clustering, namely F_U and the iterative formulations F_h^S and F_e^S.

We also study two open questions. First, we add simple variables built from the original features. Second, we use a dedicated univariate unit-flow formulation to study *opposite corners*, *linked sets*, rounding, and a simple cutting-plane strategy. Two larger datasets, `titanic` and `diabetes`, are used only in these additional experiments and in an extended exact comparison.

2 Experimental Setup

Each dataset is normalized to $[0, 1]$. We then use a fixed 80/20 train-test split with seed 1. The tested depths are $D \in \{2, 3, 4\}$. The repository does not record the full hardware and software environment, so solving times should be read as machine-dependent. All runs were still produced with the same local Julia and solver setup.

When several runs exist for the same script and dataset, the tables keep the run with the largest time limit. In `main_merge.jl` and `main_iterative_algorithm.jl`, several values of γ are tested at each depth. To keep the tables short, we keep only the configuration with the lowest test error. We then use train error and solving time as tie-breakers.

The scripts `main.jl` and `main_merge.jl` are tested in both the univariate and multivariate settings. The script `main_iterative_algorithm.jl` is tested only in the univariate case, as required in the statement. For this algorithm, the time limit applies to each subproblem, so the total time can be larger than 180.0 s. The grouped and iterative tables focus on the five datasets used for the main comparison: the three provided datasets and the two additional datasets `banknote` and `wdbc`. Short exploratory runs on `titanic` and `diabetes` are not kept in the final grouped and iterative tables. The study of opposite-corner and linked-set equalities uses a separate depth-2 setting with a 30.0 s limit per LP or MIP solve, because each reported time already includes several solves. When a positive optimality gap remains, the reported tree should be read as the best feasible solution found within the time limit, not as a proven optimum.

3 Main Results

3.1 Exact Formulation

The exact formulation is the reference for the rest of the study. Tables 1 and 2 report the results on the seven datasets. Each cell gives train and test errors, train and test accuracies, solving time, and the solver gap.

Dataset	D	Univariate	Multivariate
iris	2	train/test = 5/1 acc. tr/te = 95.8%/96.7% 3.4 s, gap 0.0%	train/test = 1/3 acc. tr/te = 99.2%/90% 0.5 s, gap 0.0%
iris	3	train/test = 0/3 acc. tr/te = 100%/90% 26.3 s, gap 0.0%	train/test = 0/3 acc. tr/te = 100%/90% 2.0 s, gap 0.0%
iris	4	train/test = 0/1 acc. tr/te = 100%/96.7% 28.5 s, gap 0.0%	train/test = 0/2 acc. tr/te = 100%/93.3% 6.2 s, gap 0.0%
seeds	2	train/test = 10/3 acc. tr/te = 94%/92.9% 66.1 s, gap 0.0%	train/test = 0/0 acc. tr/te = 100%/100% 0.5 s, gap 0.0%
seeds	3	train/test = 6/3 acc. tr/te = 96.4%/92.9% 180.0 s, gap 3.7%	train/test = 0/1 acc. tr/te = 100%/97.6% 3.3 s, gap 0.0%
seeds	4	train/test = 4/5 acc. tr/te = 97.6%/88.1% 180.1 s, gap 2.4%	train/test = 0/2 acc. tr/te = 100%/95.2% 16.5 s, gap 0.0%
wine	2	train/test = 5/2 acc. tr/te = 96.5%/94.4% 31.4 s, gap 0.0%	train/test = 0/1 acc. tr/te = 100%/97.2% 0.3 s, gap 0.0%
wine	3	train/test = 0/2 acc. tr/te = 100%/94.4% 56.0 s, gap 0.0%	train/test = 0/0 acc. tr/te = 100%/100% 1.1 s, gap 0.0%
wine	4	train/test = 2/4 acc. tr/te = 98.6%/88.9% 180.1 s, gap 1.4%	train/test = 0/2 acc. tr/te = 100%/94.4% 9.4 s, gap 0.0%
banknote	2	train/test = 390/100 acc. tr/te = 64.4%/63.6% 181.1 s, gap 14.8%	train/test = 0/1 acc. tr/te = 100%/99.6% 3.9 s, gap 0.0%
banknote	3	train/test = 382/103 acc. tr/te = 65.2%/62.5% 180.2 s, gap 22.0%	train/test = 0/0 acc. tr/te = 100%/100% 6.7 s, gap 0.0%
banknote	4	train/test = 391/102 acc. tr/te = 64.4%/62.9% 180.5 s, gap 23.5%	train/test = 605/157 acc. tr/te = 44.8%/42.9% 181.6 s, gap 123.0%
wdbc	2	train/test = 20/11 acc. tr/te = 95.6%/90.4% 180.1 s, gap 4.6%	train/test = 0/6 acc. tr/te = 100%/94.7% 4.2 s, gap 0.0%
wdbc	3	train/test = 18/14 acc. tr/te = 96%/87.7% 180.3 s, gap 4.1%	train/test = 0/5 acc. tr/te = 100%/95.6% 19.1 s, gap 0.0%
wdbc	4	train/test = 25/13 acc. tr/te = 94.5%/88.6% 180.2 s, gap 6.1%	train/test = 169/43 acc. tr/te = 62.9%/62.3% 181.8 s, gap 59.1%

Table 1: Results of `main.jl` on `iris`, `seeds`, `wine`, `banknote`, and `wdbc`.

Dataset	D	Univariate	Multivariate
titanic	2	train/test = 137/38 acc. tr/te = 80.8%/78.8% 360.3 s, gap 23.8%	train/test = 125/39 acc. tr/te = 82.4%/78.2% 360.8 s, gap 21.3%
titanic	3	train/test = 118/32 acc. tr/te = 83.4%/82.1% 360.3 s, gap 19.9%	train/test = 141/38 acc. tr/te = 80.2%/78.8% 360.3 s, gap 28.5%
titanic	4	train/test = 163/45 acc. tr/te = 77.1%/74.9% 361.2 s, gap 29.7%	train/test = 263/79 acc. tr/te = 63.1%/55.9% 360.7 s, gap 58.6%
diabetes	2	train/test = 0/0 acc. tr/te = 100%/100% 65.2 s, gap 0.0%	train/test = 0/6 acc. tr/te = 100%/99.5% 29.8 s, gap 0.0%
diabetes	3	train/test = 3880/967 acc. tr/te = 19.2%/19.4% 361.3 s, gap 4.8e9%	train/test = 0/6 acc. tr/te = 100%/99.5% 19.3 s, gap 0.0%
diabetes	4	train/test = 2719/679 acc. tr/te = 43.4%/43.4% 362.5 s, gap 130.7%	train/test = 3880/967 acc. tr/te = 19.2%/19.4% 362.5 s, gap 421.7%

Table 2: Results of `main.jl` on `titanic` and `diabetes`.

The main trend is clear on the first five datasets. On `seeds`, `wine`, and `banknote`, the multivariate formulation is usually better at depths 2 and 3. It is much faster and often reaches test accuracies between 97% and 100% on the kept runs. On `iris`, the difference is smaller, but the multivariate version is still faster. On `wdbc`, it improves test accuracy at depths 2 and 3, then becomes much worse at depth 4. The two larger datasets give more mixed results. On `titanic`, both variants are expensive and the best result changes with depth. On `diabetes`, the depth-2 univariate run reaches 100% train and test accuracy, but deeper univariate models become unstable. When the remaining gap is large, these values should be read as incumbent performances only.

3.2 Grouped Formulation

The script `main_merge.jl` replaces individual samples with same-class clusters that cannot be split anymore. Tables 3 and 4 keep only the best value of γ at each depth. So, each column gives the best configuration found at that depth.

Dataset	D	Univariate	Multivariate
iris	2	$\gamma = 20\%$ train/test = 5/1 acc. tr/te = 95.8%/96.7% 0.1 s, gap 0.0%	$\gamma = 20\%$ train/test = 1/0 acc. tr/te = 99.2%/100% 0.5 s, gap 0.0%
iris	3	$\gamma = 40\%$ train/test = 0/1 acc. tr/te = 100%/96.7% 2.7 s, gap 0.0%	$\gamma = 100\%$ train/test = 0/0 acc. tr/te = 100%/100% 1.8 s, gap 0.0%
iris	4	$\gamma = 40\%$ train/test = 0/1 acc. tr/te = 100%/96.7% 1.8 s, gap 0.0%	$\gamma = 100\%$ train/test = 0/1 acc. tr/te = 100%/96.7% 14.0 s, gap 0.0%
seeds	2	$\gamma = 60\%$ train/test = 12/2 acc. tr/te = 92.9%/95.2% 2.2 s, gap 0.0%	$\gamma = 40\%$ train/test = 0/0 acc. tr/te = 100%/100% 0.5 s, gap 0.0%
seeds	3	$\gamma = 60\%$ train/test = 5/2 acc. tr/te = 97%/95.2% 180.0 s, gap 1.8%	$\gamma = 40\%$ train/test = 0/1 acc. tr/te = 100%/97.6% 1.9 s, gap 0.0%
seeds	4	$\gamma = 60\%$ train/test = 4/2 acc. tr/te = 97.6%/95.2% 180.1 s, gap 1.8%	$\gamma = 80\%$ train/test = 0/0 acc. tr/te = 100%/100% 34.1 s, gap 0.0%
wine	2	$\gamma = 60\%$ train/test = 5/2 acc. tr/te = 96.5%/94.4% 2.1 s, gap 0.0%	$\gamma = 100\%$ train/test = 0/1 acc. tr/te = 100%/97.2% 0.7 s, gap 0.0%
wine	3	$\gamma = 80\%$ train/test = 0/2 acc. tr/te = 100%/94.4% 51.2 s, gap 0.0%	$\gamma = 0\%$ train/test = 0/1 acc. tr/te = 100%/97.2% 0.1 s, gap 0.0%
wine	4	$\gamma = 60\%$ train/test = 0/3 acc. tr/te = 100%/91.7% 44.2 s, gap 0.0%	$\gamma = 80\%$ train/test = 0/0 acc. tr/te = 100%/100% 7.7 s, gap 0.0%

Table 3: Best configurations of `main_merge.jl` on `iris`, `seeds`, and `wine`.

Dataset	D	Univariate	Multivariate
banknote	2	$\gamma = 40\%$ train/test = 399/102 acc. tr/te = 63.6%/62.9% 59.1 s, gap 0.0%	no feasible solution within 180.0 s
banknote	3	$\gamma = 20\%$ train/test = 414/102 acc. tr/te = 62.3%/62.9% 19.6 s, gap 0.0%	no feasible solution within 180.0 s
banknote	4	$\gamma = 0\%$ train/test = 492/118 acc. tr/te = 55.2%/57.1% 1.0 s, gap 0.0%	no feasible solution within 180.0 s
wdbc	2	$\gamma = 80\%$ train/test = 18/6 acc. tr/te = 96%/94.7% 180.1 s, gap 4.1%	$\gamma = 20\%$ train/test = 0/6 acc. tr/te = 100%/94.7% 0.6 s, gap 0.0%
wdbc	3	$\gamma = 80\%$ train/test = 16/4 acc. tr/te = 96.5%/96.5% 523.7 s, gap 3.9%	$\gamma = 0\%$ train/test = 0/6 acc. tr/te = 100%/94.7% 1.6 s, gap 0.0%
wdbc	4	$\gamma = 80\%$ train/test = 29/6 acc. tr/te = 93.6%/94.7% 180.3 s, gap 7.1%	$\gamma = 40\%$ train/test = 0/4 acc. tr/te = 100%/96.5% 36.3 s, gap 0.0%

Table 4: Best configurations of `main_merge.jl` on `banknote` and `wdbc`.

This grouped formulation stays reasonable in the univariate setting on the three smaller datasets. It often reaches error levels close to the exact univariate approach, with lower times at depths 2 and 3. In the multivariate setting, it is strong on `iris`, `seeds`, `wine`, and `wdbc`. For the kept values of γ, the grouped trees reach zero training error on all four datasets and often test accuracies above 97% in a few seconds. The clear exception is `banknote`, where no multivariate grouped run found a feasible solution within 180.0 s. The short exploratory grouped runs on `titanic` and `diabetes` are not discussed here.

3.3 Iterative Algorithms

The script `main_iterative_algorithm.jl` is studied only in the univariate case. Tables 5 and 6 compare the best versions of F_h^S, F_h^S with *shifts*, and F_e^S. Each cell gives train and test errors, accuracies, total time, and gap.

Dataset	D	F_h^S	F_h^S + shifts	F_e^S
iris	2	$\gamma = 0\%$ train/test = 5/1 acc. tr/te = 95.8%/96.7% 1.8 s, gap 0.0%	$\gamma = 0\%$ train/test = 5/1 acc. tr/te = 95.8%/96.7% 1.5 s, gap 0.0%	$\gamma = 60\%$ train/test = 5/1 acc. tr/te = 95.8%/96.7% 8.9 s, gap 0.0%
iris	3	$\gamma = 20\%$ train/test = 0/1 acc. tr/te = 100%/96.7% 20.8 s, gap 0.0%	$\gamma = 0\%$ train/test = 0/0 acc. tr/te = 100%/100% 9.0 s, gap 0.0%	$\gamma = 80\%$ train/test = 0/3 acc. tr/te = 100%/90% 19.5 s, gap 0.0%
iris	4	$\gamma = 0\%$ train/test = 0/1 acc. tr/te = 100%/96.7% 4.4 s, gap 0.0%	$\gamma = 40\%$ train/test = 0/0 acc. tr/te = 100%/100% 18.5 s, gap 0.0%	$\gamma = 80\%$ train/test = 0/1 acc. tr/te = 100%/96.7% 66.7 s, gap 0.0%
seeds	2	$\gamma = 40\%$ train/test = 10/2 acc. tr/te = 94%/95.2% 14.0 s, gap 0.0%	$\gamma = 40\%$ train/test = 10/2 acc. tr/te = 94%/95.2% 11.3 s, gap 0.0%	$\gamma = 60\%$ train/test = 10/2 acc. tr/te = 94%/95.2% 66.3 s, gap 0.0%
seeds	3	$\gamma = 80\%$ train/test = 7/1 acc. tr/te = 95.8%/97.6% 180.1 s, gap 2.4%	$\gamma = 40\%$ train/test = 0/0 acc. tr/te = 100%/100% 180.0 s, gap n/a	$\gamma = 60\%$ train/test = 9/2 acc. tr/te = 94.6%/95.2% 180.1 s, gap 1.8%
seeds	4	$\gamma = 80\%$ train/test = 2/2 acc. tr/te = 98.8%/95.2% 180.2 s, gap 1.2%	$\gamma = 60\%$ train/test = 0/0 acc. tr/te = 100%/100% 180.1 s, gap 0.6%	$\gamma = 80\%$ train/test = 4/2 acc. tr/te = 97.6%/95.2% 180.1 s, gap 1.8%
wine	2	$\gamma = 0\%$ train/test = 5/2 acc. tr/te = 96.5%/94.4% 9.9 s, gap 0.0%	$\gamma = 40\%$ train/test = 0/0 acc. tr/te = 100%/100% 26.0 s, gap 0.0%	$\gamma = 80\%$ train/test = 5/2 acc. tr/te = 96.5%/94.4% 69.0 s, gap 0.0%
wine	3	$\gamma = 0\%$ train/test = 3/1 acc. tr/te = 97.9%/97.2% 180.0 s, gap n/a	$\gamma = 20\%$ train/test = 0/0 acc. tr/te = 100%/100% 29.7 s, gap 0.0%	$\gamma = 80\%$ train/test = 0/3 acc. tr/te = 100%/91.7% 22.3 s, gap 0.0%
wine	4	$\gamma = 20\%$ train/test = 0/2 acc. tr/te = 100%/94.4% 21.8 s, gap 0.0%	$\gamma = 20\%$ train/test = 0/0 acc. tr/te = 100%/100% 36.8 s, gap 0.0%	$\gamma = 80\%$ train/test = 14/5 acc. tr/te = 90.1%/86.1% 180.1 s, gap 1.4%

Table 5: Best configurations of `main_iterative_algorithm.jl` on `iris`, `seeds`, and `wine`.

Dataset	D	F_h^S	F_h^S + shifts	F_e^S
banknote	2	$\gamma = 0\%$ train/test = 136/42 acc. tr/te = 87.6%/84.7% 12.7 s, gap 0.0%	$\gamma = 0\%$ train/test = 136/42 acc. tr/te = 87.6%/84.7% 11.1 s, gap 0.0%	$\gamma = 0\%$ train/test = 166/49 acc. tr/te = 84.9%/82.2% 180.5 s, gap 0.2%
banknote	3	$\gamma = 0\%$ train/test = 149/45 acc. tr/te = 86.4%/83.6% 180.0 s, gap n/a	$\gamma = 0\%$ train/test = 0/0 acc. tr/te = 100%/100% 180.0 s, gap n/a	$\gamma = 0\%$ train/test = 171/46 acc. tr/te = 84.4%/83.3% 973.9 s, gap 2.8%
banknote	4	$\gamma = 0\%$ train/test = 215/40 acc. tr/te = 80.4%/85.5% 180.4 s, gap 9.4%	$\gamma = 0\%$ train/test = 0/0 acc. tr/te = 100%/100% 180.5 s, gap n/a	$\gamma = 0\%$ train/test = 311/74 acc. tr/te = 71.6%/73.1% 180.6 s, gap 12.3%
wdbc	2	$\gamma = 40\%$ train/test = 28/5 acc. tr/te = 93.8%/95.6% 180.2 s, gap 1.8%	$\gamma = 40\%$ train/test = 28/5 acc. tr/te = 93.8%/95.6% 180.2 s, gap 1.8%	$\gamma = 40\%$ train/test = 38/8 acc. tr/te = 91.6%/93% 180.2 s, gap 1.6%
wdbc	3	$\gamma = 20\%$ train/test = 41/9 acc. tr/te = 91%/92.1% 1033.3 s, gap 0.4%	$\gamma = 40\%$ train/test = 0/0 acc. tr/te = 100%/100% 180.4 s, gap 0.7%	$\gamma = 20\%$ train/test = 28/6 acc. tr/te = 93.8%/94.7% 180.3 s, gap n/a
wdbc	4	$\gamma = 40\%$ train/test = 24/4 acc. tr/te = 94.7%/96.5% 180.4 s, gap 1.3%	$\gamma = 0\%$ train/test = 0/0 acc. tr/te = 100%/100% 180.8 s, gap 1.6%	$\gamma = 80\%$ train/test = 53/15 acc. tr/te = 88.4%/86.8% 181.5 s, gap 10.2%

Table 6: Best configurations of `main_iterative_algorithm.jl` on `banknote` and `wdbc`.

The main result here is the role of *shifts*. The F_h^S variant with *shifts* is the most regular one. It reaches 100% test accuracy several times on `iris`, `seeds`, and `wine`, and also on `wdbc` at larger depths. The plain

F_h^S version is useful but less stable. The exact version F_e^S is rarely the best compromise. It is usually slower and does not bring a clear gain in test accuracy. As in the grouped study, the short exploratory iterative runs on `titanic` and `diabetes` are not kept in the final comparison tables.

4 Additional Work

4.1 Feature Augmentation

4.1.1 Algorithm for Feature Generation and Selection

We address open question 3 with a simple heuristic, in the spirit of the course. On the three base datasets (`iris`, `seeds`, and `wine`), the script `test_q3.jl` builds candidate variables from every pair of original features and keeps only simple transformations:

1. **Generation:** for each pair (X_a, X_b), generate $X_a + X_b$, $|X_a - X_b|$, $X_a X_b$, X_a/X_b, and X_b/X_a.
2. **Scoring:** evaluate each candidate on the training split only with the absolute Pearson correlation obtained after encoding the classes by integers.
3. **Selection:** append only the two best candidates ($k = 2$) to keep the formulation small and the added variables interpretable.

This score is only a ranking heuristic. On multiclass datasets, it should not be read as a strong statistical measure by itself. The real evaluation is the tree performance after the variables are added.

For the two larger additional datasets, we use precomputed augmented versions of `titanic` and `diabetes`. In these two cases, the script only compares the original and augmented datasets. It does not regenerate or rerank the extra variables inside the current pipeline.

4.1.2 Generated Features on the Three Base Datasets

Dataset	Selected generated features	Heuristic scores
iris	$x_2 + x_4$, $x_3 \times x_4$	0.529, 0.502
seeds	$x_1 + x_4$, $x_3 + x_4$	0.908, 0.901
wine	$x_2 + x_{10}$, $x_2 \times x_{10}$	0.786, 0.739

Table 7: Two generated variables retained by the current Q3 pipeline on the three base datasets.

4.1.3 Impact on Classification Performance

Table 8 reports the kept depth-2 comparisons. The first three rows correspond to the on-the-fly feature generation procedure above, with two added variables and a 60.0 s limit. The last two rows correspond to the precomputed augmented versions of `titanic` and `diabetes`, which add three variables and were evaluated with a 10.0 s limit.

The effect stays limited. On `iris`, the two generated variables improve the training fit but require one more active split node and slightly worsen the test result. On `seeds`, the change is small and does not reduce the tree size. On `wine`, the generated variables improve the training error while keeping the same depth-2 structure. On `titanic`, the precomputed augmented dataset does not improve the result. It keeps the same number of split nodes and slightly worsens both train and test errors. On `diabetes`, the augmented version does not change the result. This study stays modest: simple extra variables can move the errors a little, but they do not reduce the tree size in the kept runs.

Dataset	Original data	Augmented version
iris	train/test = 5/1 acc. tr/te = 95.8%/96.7% split nodes = 2 11.7 s	train/test = 2/2 acc. tr/te = 98.3%/93.3% split nodes = 3 4.3 s
seeds	train/test = 11/2 acc. tr/te = 93.5%/95.2% split nodes = 3 60.1 s	train/test = 10/3 acc. tr/te = 94%/92.9% split nodes = 3 47.0 s
wine	train/test = 7/2 acc. tr/te = 95.1%/94.4% split nodes = 3 60.6 s	train/test = 5/2 acc. tr/te = 96.5%/94.4% split nodes = 3 60.0 s
titanic	train/test = 150/39 acc. tr/te = 78.9%/78.2% split nodes = 3 11.7 s	train/test = 165/40 acc. tr/te = 76.8%/77.7% split nodes = 3 10.1 s
diabetes	train/test = 3880/967 acc. tr/te = 19.2%/19.4% split nodes = 3 11.0 s	train/test = 3880/967 acc. tr/te = 19.2%/19.4% split nodes = 3 11.0 s

Table 8: Effect of feature augmentation at depth 2. Each cell reports train/test errors, the corresponding train/test accuracies, the number of active split nodes, and the solving time.

4.2 Opposite Corners, Linked Sets, and Rounding

The second additional study is implemented in `main_q4_opposite_corner.jl`. It uses a dedicated univariate formulation at depth 2. In the exact formulation used in the rest of the report, a sample carries flow only when it is correctly classified. The opposite-corner and linked-set equalities are valid only in a unit-flow setting. So the script builds a separate model in which every training sample starts with one unit of flow at the root, follows exactly one root-to-terminal-node path, and contributes to the objective only if it is finally assigned to its true class.

The code keeps the same split variables as in the univariate OCT model: a binary feature-selection variable and a threshold at each internal node, together with node-label variables. It also introduces three flow families. The variable u_t^i is equal to 1 when sample i reaches node t. The variable v_t^i is equal to 1 when the path of sample i stops at node t. The variable $z_{t,k}^i$ is equal to 1 when sample i is assigned class k at node t. Flow conservation forces a unique root-to-terminal-node path. Stopping is allowed only at labeled nodes. The usual univariate strict-separation constraints are kept through the μ constants computed from the training data, as in the course formulation.

The candidate equalities are generated directly from the training set. For opposite corners, the script scans all pairs of samples, builds the axis-aligned box defined by the two points, keeps only boxes that vary on at least two features, and checks which corners of this box are present in the dataset. Two pairs of opposite corners then define one candidate equality. For linked sets, the script fixes one active feature $\bar{j}$, groups the samples that have the same coordinates on all other features, orders these samples along $\bar{j}$, and creates the cyclic equality only when at least three distinct values are present with repeated copies. This is exactly the combinatorial rule used in the code. No external geometric preprocessing is added.

At depth 2, if (i_1, i_2, i_3, i_4) is an opposite-corner candidate, the relation for a child node k of the root is

$$u_k^{i_1} + u_k^{i_2} = u_k^{i_3} + u_k^{i_4}.$$

In the implementation, it is enough to impose this equality on one child of the root, because under unit flow the equality on the other child is then fixed automatically. The script chooses the left child, that is,

node 2 in the depth-2 tree. For a linked-set candidate (P, Q), the code writes, for every class c,

$$\sum_{i \in P} \sum_{t \in V} z^i_{t,c} = \sum_{i \in Q} \sum_{t \in V} z^i_{t,c},$$

where V is the set of tree nodes. So the total terminal assignment of P and Q must match class by class.

Figure 1 shows the opposite-corner equality at depth 2. In the left panel, the two diagonals identify the two opposite pairs (i_1, i_2) and (i_3, i_4) of the same box. In the right panel, the child under study is called k, and the other child is called $\bar{k}$. When the root splits on feature $\bar{j}$, each opposite pair sends the same number of samples to node k, as in the course slides.

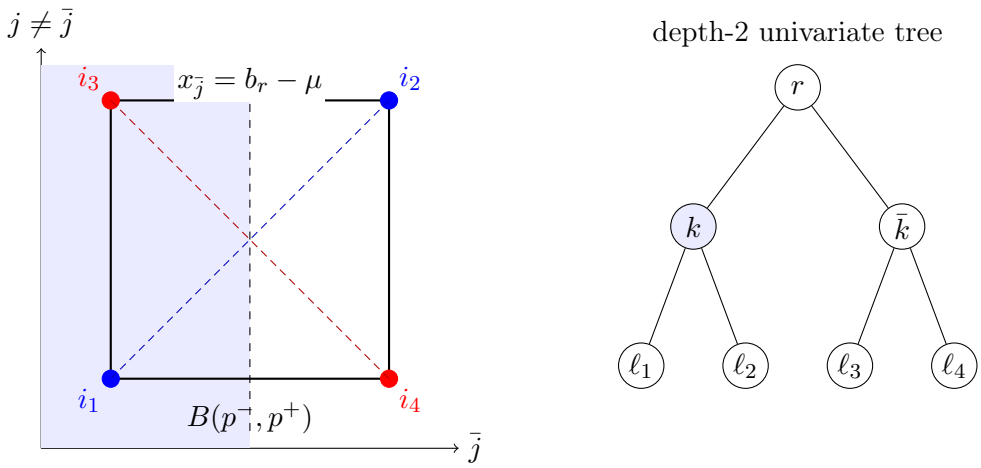

Figure 1: Opposite-corner equality at depth 2. The four samples are corners of the same box. If the root splits on feature $\bar{j}$, the number of samples from (i_1, i_2) sent to node k is equal to the number of samples from (i_3, i_4) sent to node k.

Figure 2 gives an explicit linked set with six points. Here $n_S = 3$ and $\bar{j}$ is the horizontal feature. The dashed horizontal lines show that each pair (p_k, q_k) has the same coordinates on all features $j \neq \bar{j}$. The dotted vertical guides mark the three distinct values of the active coordinate. The linked-set condition is read only on the horizontal coordinates:

$$x^{q_1}_{\bar{j}} = x^{p_2}_{\bar{j}}, \qquad x^{q_2}_{\bar{j}} = x^{p_3}_{\bar{j}}, \qquad x^{q_3}_{\bar{j}} = x^{p_1}_{\bar{j}}.$$

In other words, the $\bar{j}$-coordinates of Q are a cyclic shift of those of P. The implementation uses a stricter special case of this construction. For each linked-set candidate generated by the code, all points of $P \cup Q$ have identical coordinates on every feature $j \neq \bar{j}$.

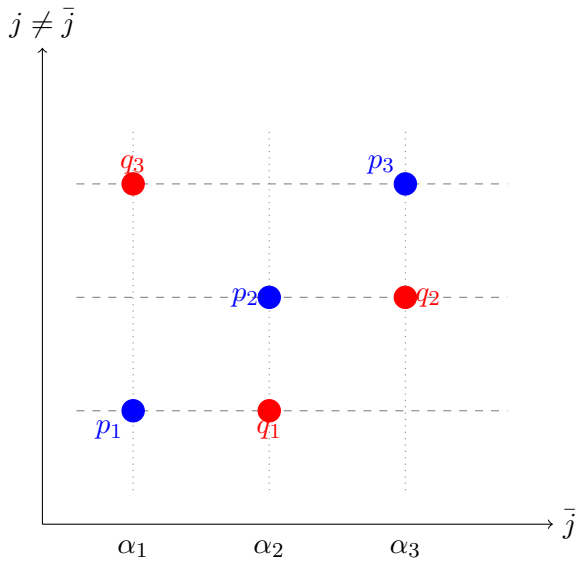

Figure 2: Example of a linked set with six points. The three blue points form $P = \{p_1, p_2, p_3\}$ and the three red points form $Q = \{q_1, q_2, q_3\}$. Each dashed horizontal line fixes the coordinates on all features $j \neq \bar{j}$, while the three dotted vertical guides correspond to the active-coordinate values $\alpha_1, \alpha_2, \alpha_3$ and make the cyclic shift visible.

Let $u^i_{\ell,w}$ denote the unit flow of sample i on arc (ℓ, w), where $\ell \in L$ is a leaf and w is the sink node.

Proposition. For each linked set $(P, Q, \bar{j})$ with $P = \{p_1, \ldots, p_{n_S}\}$ and $Q = \{q_1, \ldots, q_{n_S}\}$,

$$\sum_{\ell \in L} \sum_{k=1}^{n_S} \left(u^{p_k}_{\ell,w} - u^{q_k}_{\ell,w} \right) = 0.$$

Proof. In the adapted unit-flow formulation, every sample starts with one unit of flow at the root, and exactly one unit leaves the tree through an arc (ℓ, w) with $\ell \in L$. So, for every sample i,

$$\sum_{\ell \in L} u^i_{\ell,w} = 1.$$

Applying this identity to all samples of P and Q yields

$$\sum_{\ell \in L} \sum_{k=1}^{n_S} \left(u^{p_k}_{\ell,w} - u^{q_k}_{\ell,w} \right) = \sum_{k=1}^{n_S} \sum_{\ell \in L} u^{p_k}_{\ell,w} - \sum_{k=1}^{n_S} \sum_{\ell \in L} u^{q_k}_{\ell,w} = \sum_{k=1}^{n_S} 1 - \sum_{k=1}^{n_S} 1 = n_S - n_S = 0.$$

This proves the proposition.

This argument shows why the unit-flow adaptation is essential, as in the course slides for the opposite-corner property. In the implemented MIP, the linked-set constraints are written in a stronger form on the terminal assignment variables $z^i_{t,k}$. For every class k, the total assignment of the samples in P must be equal to the total assignment of the samples in Q. This is stronger than the proposition because it balances assignments class by class, not only total flow.

The script then evaluates three variants of this formulation. The *baseline* solves the unit-flow model without extra equalities. The *all equalities* variant adds every generated opposite-corner and linked-set equality from the start. The *cutting-plane* variant first solves the LP relaxation, finds which candidate equalities are violated by the current fractional solution, adds only those violated equalities, and repeats this process until no further violated candidate is found. Only then is the final integer model solved. For this last variant, the reported time is the total algorithm time, that is, the sum of all LP rounds and the final MIP solve.

The code reports three experiments. First, it compares the three variants on the raw normalized training set. Second, because no useful equality is generated on the original versions of `iris`, `seeds`, and `wine`, it

reruns the same comparison on the first rounded version that produces at least one candidate equality. Third, it isolates the effect of the rounding level by keeping only the cutting-plane strategy and testing several rounding precisions.

Dataset	Rounding	Mode	Added eq.	LP value	Train err.	Time (s)	B&B nodes
iris	1	baseline	0 / 0	120.0	5	12.5	1606
iris	1	all equalities	237 / 4	120.0	5	8.1	317
iris	1	cutting plane	186 / 0	120.0	5	10.0	433
seeds	1	baseline	0 / 0	168.0	16	28.7	5409
seeds	1	all equalities	60 / 0	168.0	16	30.1	5406
seeds	1	cutting plane	44 / 0	168.0	16	30.7	5668
wine	0	baseline	0 / 0	142.0	21	15.3	139
wine	0	all equalities	1421 / 0	142.0	21	8.8	0
wine	0	cutting plane	1028 / 0	142.0	21	9.3	0

Table 9: Comparison of the unit-flow formulation, all equalities, and the cutting-plane variant on the first useful rounded version of each dataset. Added equalities are reported as opposite-corner / linked-set counts.

Two remarks stand out. First, the LP objective stays equal to the number of training samples in all tested cases. So the unit-flow relaxation remains weak on these datasets, and the added equalities do not improve the root bound in the reported runs. Second, the effect on the search tree is mixed. On `iris`, the opposite-corner equalities clearly reduce the number of branch-and-bound nodes and the full model is the fastest one. On `seeds`, the generated equalities do not improve the LP objective or the training error, and they slightly increase total time. On `wine`, the equalities become useful only after very coarse rounding. Then they remove the search tree, but the rounded data are already much less faithful to the original geometry.

Dataset	Rounding	Candidates	Added eq.	Train err.	Time (s)	B&B nodes
iris	none	0 / 0	0 / 0	5	8.3	1223
iris	2	0 / 0	0 / 0	5	15.7	2170
iris	1	237 / 4	186 / 0	5	8.3	433
iris	0	1333 / 0	741 / 0	22	2.9	0
seeds	none	0 / 0	0 / 0	11	30.5	4921
seeds	2	0 / 0	0 / 0	11	30.4	4655
seeds	1	60 / 0	44 / 0	16	24.5	5663
seeds	0	2866 / 0	556 / 0	25	4.5	0
wine	none	0 / 0	0 / 0	5	30.4	3593
wine	2	0 / 0	0 / 0	8	30.5	3057
wine	1	0 / 0	0 / 0	7	20.8	2270
wine	0	1421 / 0	1028 / 0	21	8.8	0

Table 10: Effect of rounding with the cutting-plane strategy. Candidate and added equalities are reported as opposite-corner / linked-set counts.

The rounding study confirms that these equalities are very sensitive to the data geometry. Mild rounding helps on `iris`: at one decimal, the training error is unchanged and the search tree becomes much smaller. The same behavior is not seen on `seeds`, where rounding hurts the training fit and does not simplify the branch-and-bound search. On `wine`, no useful equality appears before rounding to zero decimals, and this strong modification sharply degrades the training fit even though the model then solves at the root. Linked sets remain rare in practice. Only `iris` at one decimal generates them, and none of these four linked-set equalities is violated by the LP solution. In this study, the visible algorithmic effect therefore comes from opposite-corner equalities only.

5 Conclusion

The exact multivariate formulation is the best practical reference on most datasets, especially at depths 2 and 3. The exact univariate formulation becomes more expensive when depth increases, with less stable behavior on `titanic` and `diabetes`.

The grouped formulation F_U is useful as a univariate approximation on medium-sized datasets. Its multivariate version is also competitive on four of the five datasets kept for the core comparison, with `banknote` as the main exception. In the iterative part, the most convincing variant in these experiments is F_h^S with *shifts*. It is the variant that most often gives the best compromise in test error on the kept runs.

The two additional studies are interesting but secondary. Adding simple variables does not bring a general improvement, even after extending the analysis to `titanic` and `diabetes`. It also does not reduce the depth-2 tree size in the kept runs. The geometric equalities require a dedicated unit-flow formulation. On the original data, they are almost absent. After rounding, their effect depends strongly on how much the data are changed. They can reduce the search tree on `iris` and on heavily rounded `wine`, but they do not improve the LP objective at depth 2 and they can easily damage the training fit. Overall, the clearest results in this report come from the comparison between the exact, grouped, and iterative formulations.